# 귀갓길 놀이를 터치 게임으로

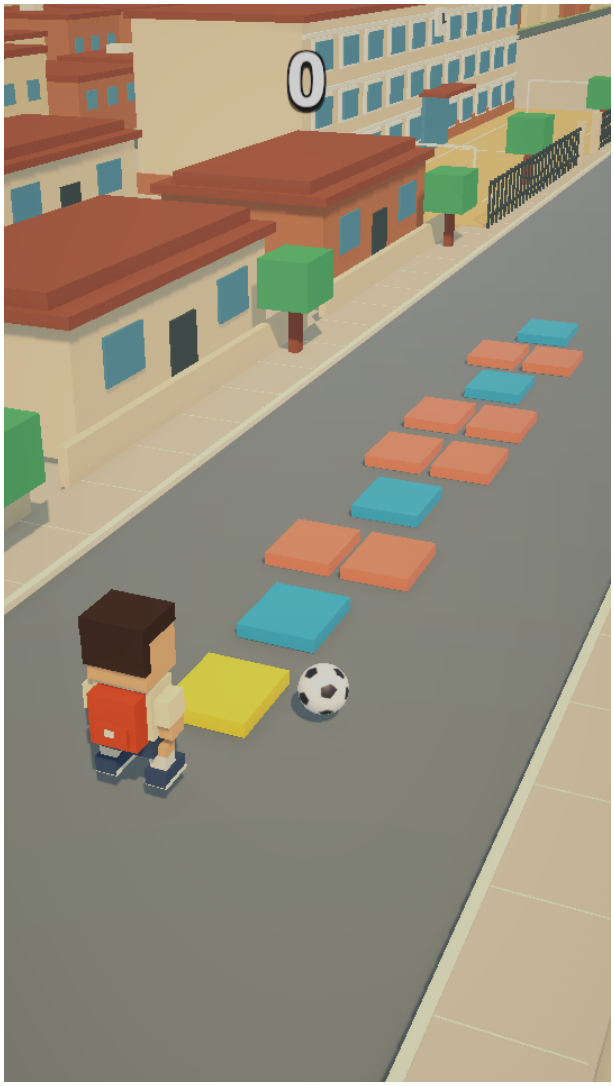
Go To Home | 세로 화면 기반 3D 캐주얼 게임

## 프로젝트 소개

학교에서 집으로 돌아가는 길에 하던 놀이를 세 가지 짧은 게임 구간으로 연결했습니다. 화면 어디서나 가능한 탭 입력으로 리듬, 균형, 거리 판단을 번갈아 경험하도록 구성했습니다.

## 검증하려는 재미

복잡한 조작 없이도 구간마다 다른 판단을 만들 수 있는가? 반복 테스트에서 발견한 입력 불편과 장애물 회피 문제를 수정하며 플레이 감각을 다듬었습니다.




**3개 게임 모드 / 최대 1.4배속 / PlayMode 52개 통과**

Unity 6000.5.0f1 · C# · URP · Input System · uGUI
대상 플랫폼: Android 세로 화면 / 현재 검증 빌드: Windows PC

자료 범위: 현재 소스와 자동 테스트 결과, 실제 빌드 화면, 제작 중 기록된 플레이 피드백. 이미지에는 진단 모드로 촬영한 정적 장면이 포함됩니다.

# 역할과 개발 방식

기획 판단, 구현 지원, 검증 범위를 구분해 기록

| 구분 | 수행 내용 |
| --- | --- |
| 제작자 foxra1n | 소재·조작 규칙·비주얼 방향 제안, 직접 플레이 테스트, 문제 재현과 수정 요구, 개선 결과의 체감 확인 |
| AI 개발 도구 지원 | C# 구현 및 수정, 구조 제안, 자동 테스트 작성·실행, PC 빌드, 문서 초안 작성 |
| 현재 기여 표기 | AI 지원을 활용한 개인 프로젝트. 전체 코드를 직접 작성한 프로젝트로 표기하지 않으며, 기능 의사결정과 구현 지원 범위를 구분한다. |

## 작업 루프

규칙 제안 → 최소 기능 구현 → 직접 플레이 → 불편한 상황 재현 → 입력·판정·연출 수정 → 자동 회귀 테스트 → PC 빌드에서 재확인.

## 범위를 작게 유지

상점, 광고, 로그인, 온라인 기능은 만들지 않았습니다. 먼저 세 가지 구간의 조작과 실패 판정, 전환과 난이도 변화를 검증하는 데 집중했습니다.

## 검증의 한계

직접 플레이 피드백은 제작자 중심의 정성적 관찰입니다. 다수 사용자 실험이나 성공률 개선 통계는 아직 없습니다. 최신 변경은 PC에서 검증했으며, 이전 Android 실행 확인과 최신 모바일 회귀 검증을 구분합니다.

# 게임 규칙을 나눈 코드 구조

입력, 진행 상태, 개별 모드, 표현을 분리

### 입력과 세션

TapInputReader → GameSessionController
터치·마우스·Space 입력 / 시작 대기·카운트다운·플레이·게임오버

### 공통 구간 진행

GameModeSequenceController + ISectionGameMode
구간 준비·시작·종료 / 다음 구간 선생성 / 이벤트 전달

### 세 가지 규칙

HopscotchModeController / CurbBalanceModeController / CrosswalkModeController
각 모드가 탭 의미와 성공·실패 조건을 독립적으로 판단

### 점수·난이도·표현

ScoreController · DifficultyController · RhythmClock · PlayerHopController
점수 저장 / 세트 기반 배율 / 오디오 박자 / 이동과 착지 연출

## 이 구조를 선택한 이유

GameSessionController가 사방치기 규칙을 직접 알지 않도록 이벤트와 인터페이스로 연결했습니다. 모드별 문제를 다른 구간에 미치는 영향을 줄여 수정할 수 있습니다.

확인할 코드: Runtime/Core/IGameMode.cs, GameSessionController.cs, GameModeSequenceController.cs
구현은 현재 기능에 맞춘 소규모 구조이며, 범용 게임 프레임워크를 목표로 하지 않습니다.

# 수정 사례 01 · 횡단보도

입력 즉시성, 착지 거리, 상태별 허용 동작

## 문제

초기에는 연타 입력이 다음 점프로 예약되거나, 공중에서 재점프가 이어지는 문제가 있었습니다. 흰색 사이 간격이 좁을 때는 연타만으로 통과할 수 있어 거리 판단이 약해졌습니다.

## 수정

공중에서는 입력을 무시하고 착지 이후 새 탭만 받도록 했습니다. 흰색 길이 2.4, 간격 1.6, 점프 거리 2.56으로 조정했습니다. 흰색 끝에서 뛰면 다음 흰색의 약 40% 지점에 착지하며, 경계 보정은 최대 0.12로 제한합니다.

| 현재 상태 | 탭을 눌렀을 때 |
| --- | --- |
| 빨간불 대기 | 인도 위치에서 점프 → 차도 안에서 차량 급정거 → 실패 |
| 초록불 / 무신호 진입 | 인도에서도 즉시 점프. 첫 흰색 착지를 검사하며 다음 흰색으로 진행한 것으로 중복 계산하지 않음 |
| 흰색 위 걷기 | 즉시 점프 후 다음 흰색 착지를 검사 |
| 점프 중 / 실패 | 추가 입력을 예약하지 않음 |

## 결과와 검증

초록불·무신호 진입 점프, 빨간불 실패 1회 발생, 점프 중 연타 무시를 자동 테스트했습니다. 차량의 모든 렌더러 경계가 인도 끝보다 차도 안쪽에 있는지도 검사합니다.

확인할 코드: CrosswalkModeController.cs, CrosswalkPatternGenerator.cs, CrosswalkTimingState.cs, CrosswalkSignal.cs

# 수정 사례 02 · 공과 바람

보이는 예고와 실제 회피 행동을 연결

## 사방치기: 공이 보이는데도 피하기 어렵다

공 입력에 일반 리듬 판정까지 적용되어 빠르거나 늦게 누르면 실패했습니다. 공은 두 패턴 전에 미리 보여주고, 공 차례에는 한 번 탭하면 즉시 점프하도록 분리했습니다.

## 착지로 끝나지 않는 회피

공 칸으로 진입하는 반응 시간은 0.85초, 해당 칸에 착지한 후 다음 칸으로 빠져나가는 시간은 0.65초입니다. 두 번째 점프도 즉시 입력을 받습니다. 공은 원래 칸에서 왼쪽 화면 밖까지 계속 굴러갑니다.

## 턱 걷기: 한 번 누르면 오히려 쉬워진다

바람이 복귀를 지나치게 느리게 만들어 한 번만 반대로 누르면 안전해지는 문제가 있었습니다. 탭은 즉시 방향을 반전하고, 바람은 이후 시간에 따라 속도를 바꾸도록 수정했습니다. 최대 기울기 속도도 제한했습니다.

## 시각·청각 신호

낙엽만으로는 바람을 알기 어렵다는 피드백에 나무 흔들림과 작은 바람 소리를 추가했습니다. 거슬리던 소리는 부드럽게 조정했습니다. 공의 돌출된 장식은 제거하고 표면 무늬가 있는 축구공으로 바꿨습니다.

### 검증한 동작

공 칸에서 다시 뛰면 통과 / 머무르면 실패 / 회피 후에도 공 이동 유지
바람 중 즉시 반전 / 바람 종료와 재시작 시 상태 복구

확인할 코드: HopscotchModeController.cs, HopscotchBallObstacle.cs, CurbBalanceState.cs, CurbWindProfile.cs

# 속도와 장애물을 단계적으로

세 구간을 한 번씩 진행하면 1세트, 순서는 랜덤

| 세트 | 장애물 대상 구간 | 속도 |
|---|---|---|
| 1 | 없음 | 1.0배 |
| 2 | 랜덤 1개 | 1.0배 |
| 3 | 랜덤 2개 | 1.0배 |
| 4 | 3개 모두 | 1.0배 |
| 5 / 6 / 7 | 3개 모두 | 1.1 / 1.2 / 1.3배 |
| 8 이후 | 3개 모두 | 최대 1.4배 |

예외: 횡단보도 신호 대기는 장애물 대상으로 선정되어도 1.2배속 이상에서만 나타납니다. 따라서 초기 세트에서는 배정 개수와 실제 보이는 장애물 개수가 다를 수 있습니다.

## 음악과 이동을 같은 배율로

이동 속도와 박자는 배율만큼 높이고 점프 시간은 그만큼 줄였습니다. 음악은 재생 위치를 유지한 채 AudioSource.pitch를 바꾸며, DSP 시간 기준 박자 위치를 함께 보정합니다. 음높이도 변하는 방식입니다.

## 고속에서도 입력 유예는 보존

사방치기 입력 허용 시간과 공 반응 시간을 배율로 줄이지 않습니다. 착지 후 다음 입력 박자까지 최소 0.3초를 확보합니다. 속도 상승이 입력 인식의 불합리함으로 이어지지 않도록 분리했습니다.

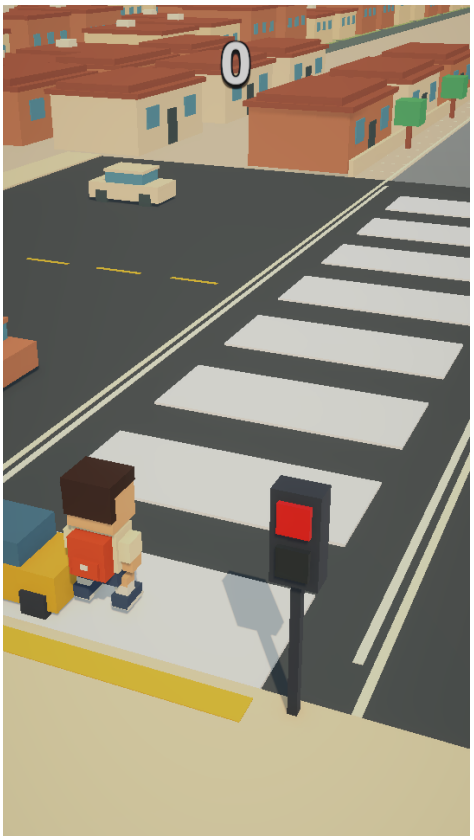
확인할 코드: DifficultyController.cs, RhythmClock.cs, GameModeShuffleBag.cs
현재 기본 실행은 정상 진행. 최대 난이도 테스트 강제 설정은 꺼져 있습니다.

# 검증 기록과 시각적 수정

테스트 통과와 실제 사용성을 구분해 평가

## PlayMode 52 / 52 통과

최근 정상 진행 복구 버전 · 2026.10.26 · 실패 0 · Windows PC 빌드 성공




## 자동 검사 범위

9세트 전환과 속도 상한, 재시작 초기화, 음악·비프음 배율, 입력 판정, 공의 두 단계 회피, 신호와 진입 점프, UI 화면 경계를 확인했습니다.

## 실제 화면 확인

PC 빌드의 진단 장면을 캡처해 차량의 차도 위치, 축구공 모양, 시작 문구와 점수 가독성을 확인했습니다. 왼쪽은 차량 급정거 위치 확인용 정적 장면입니다.

## 수치 해석

52개 통과는 지정한 조건의 회귀 검사 결과입니다. 모든 버그가 없거나 모바일 성능이 보장된다는 뜻은 아닙니다. FPS·발열·메모리 측정 결과는 아직 제시하지 않습니다.

근거: docs/verification.json. 원본 테스트 보고서의 PC 경로 등 환경 식별 정보는 제출 자료에서 제외했습니다.

# 실행과 다음 개발 과제

소스 검토와 재현을 위한 안내

## 프로젝트 실행

Unity Hub에서 저장소 루트 폴더를 추가하고 Unity 6000.5.0f1로 엽니다. 패키지 복원 후 Assets/Scenes/SampleScene.unity를 열고 Play를 누릅니다. 캐릭터·배경·UI는 실행 시 코드로 생성됩니다.

## 조작과 코드 확인

터치, 마우스 왼쪽 버튼 또는 Space를 사용합니다. 사방치기는 톡·톡톡, 턱 걷기는 균형 방향 반전, 횡단보도는 점프입니다. Tests/PlayMode와 Tests/Editor를 Unity Test Runner에서 실행할 수 있습니다.

## 남은 과제

최신 Android 빌드의 실기기 회귀 테스트와 성능 측정, 처음 접하는 사용자의 튜토리얼 테스트, 일시정지·소리 설정, 난이도와 장애물 노출 빈도 조정이 남아 있습니다. 광고·결제·온라인 기능은 현재 범위에 포함하지 않습니다.

## 개발 포트폴리오로서의 초점

기능 수보다 입력이 처리되는 시점, 상태별 판정, 재현 가능한 실패 사례와 수정 근거를 설명하는 데 초점을 두었습니다. 향후 변경도 작은 단위의 기록과 테스트로 남길 계획입니다.

### 소스와 상세 기록

github.com/foxra1n/go-to-home
비공개 저장소: 링크 열람에는 저장소 접근 권한이 필요합니다.